Attribution-Based Explanations of Neural Summary Networks in Amortized Simulation-Based Inference

Feriha Bircan

Research internship project

https://github.com/ferihabircan/bayesflow-xai

October 2026

Abstract

Amortized Bayesian inference with BayesFlow compresses raw simulator output into a learned summary vector before a conditional normalizing flow maps it to a posterior distribution. Whether this summary is built from the physically meaningful part of the data is usually assessed only indirectly, through parameter recovery and simulation-based calibration. We study four post-hoc attribution methods (Integrated Gradients, Saliency Maps, Attention Rollout and Masking/Occlusion) applied to summary-network architectures trained on four simulators of increasing complexity: a stationary SIR epidemic model, a sinusoidal toy signal with known ground truth, a Gaussian random field, and a gravitational-wave benchmark. Gradient-based attribution is made possible by surrogate regressors that share the summary network's architecture, and all combinations are run through a single configuration-driven plugin registry. On the toy signal, Integrated Gradients recovers the known ground truth in 188 of 200 samples. On the gravitational-wave benchmark it reveals that a network trained on min-max-normalized strain ignores the merger and relies on a non-physical ~20 Hz artefact introduced by the simulator's whitening step; switching to z-score normalization moves the attribution onto the merger and reduces validation error from 9.27 to 1.41. On one trained model, Integrated Gradients and Masking agree closely (rank correlation 0.99), whereas Saliency places 58% of its mass in signal-free regions. The SIR experiments delimit the approach: methods disagree on the relevant input channel, and the recurrent read-out and the attribution baseline leave identifiable fingerprints on the maps.

1 Introduction

Many scientific models are available only as stochastic simulators: given parameters θ one can sample data $x \sim p(x \mid \theta)$, but the likelihood cannot be evaluated. Simulation-based inference (SBI) [4] replaces likelihood evaluations by learning from simulated pairs (θ, x). Neural posterior estimation [9, 14], as implemented in BayesFlow [15, 17], trains a *summary network* that compresses a data set into a fixed-size vector jointly with a conditional normalizing flow that represents the posterior. Once trained, inference on a new data set requires a single forward pass; the training cost is *amortized* over all future data sets.

The reliability of such estimators is normally established with loss curves, parameter recovery and simulation-based calibration (SBC) [22]. These diagnostics show whether the posterior is accurate *on simulated data*; they do not show *which features* of the data the network relies on. A network can reach a low loss by exploiting a property of the simulator that real observations will not have, and posterior approximations can be unfaithful in ways that standard training metrics do not expose [10].

Feature-attribution methods from explainable AI (XAI) assign an importance score to every input element for a given network output [1, 19, 21, 23]. They are widely used for image classifiers but rarely for the summary networks of amortized SBI. We ask:

Q1 Do attributions recover what is known to be informative when the ground truth is known?

Q2 Do different attribution methods agree on the same trained model?

Q3 Can attributions reveal simulator-induced problems that a model's loss alone does not explain?

Contributions. (i) An open-source plugin registry in which simulators, summary networks, inference networks and attribution methods are registered by name and combined from a configuration file. (ii) A study of four attribution methods on four simulators, including a gravitational-wave benchmark from the SBI literature [6]. (iii) Evidence that attribution exposes a simulator artefact on that benchmark, which the network trained with the original normalization relies on, together with a quantitative comparison of three methods on one model.

2 Related Work

Amortized and neural SBI. Neural posterior estimation learns $q(\theta \mid x)$ directly from simulations [9, 14]; BayesFlow [15, 17] couples learned summary statistics with invertible networks [7, 8], and has been applied, among others, to epidemiological models [16]. In gravitational-wave astronomy, neural posterior estimation enables real-time parameter inference [5]. Validation relies on calibration checks such as SBC [22], and Hermans et al. [10] showed that SBI posteriors can be overconfident, motivating additional diagnostics.

Feature attribution. Gradient-based methods range from raw input gradients [19] to path-integrated gradients [21]; perturbation-based methods occlude input regions [23]; model-agnostic surrogates include LIME [18] and SHAP [13]. For Transformers, attention rollout prop-

agates attention through layers [1]. Several studies show that attributions can be unreliable: saliency maps may be insensitive to the model parameters [2], depend on input shifts [11], and, for path methods, strongly on the choice of baseline [20]. Ancona et al. [3] relate gradient-based methods formally. Our study takes these caveats into the SBI setting and uses agreement between methods and known ground truth as sanity checks.

3 Background

3.1 Amortized posterior estimation

Let s_ψ denote the summary network and $q_\phi(\theta \mid s)$ a conditional normalizing flow. BayesFlow minimizes the expected negative log posterior density over the joint distribution of parameters and data,

$$\min_{\psi,\phi} \mathbb{E}_{p(\theta)\,p(x|\theta)}\left[-\log q_\phi\left(\theta \,\middle|\, s_\psi(x)\right)\right]. \tag{1}$$

At the optimum, $q_\phi(\theta \mid s_\psi(x))$ equals the true posterior provided that $s_\psi(x)$ is a sufficient summary. *Parameter recovery* compares posterior point estimates with the ground truth on held-out simulations; *SBC* checks that the rank of the true parameter among posterior draws is uniform, as it must be for a calibrated posterior [22].

3.2 Attribution methods

Let $f : \mathbb{R}^d \to \mathbb{R}$ be a scalar model output and x' a baseline input.

Saliency [19] is the absolute input gradient,

$$S_i(x) = |\partial f(x)/\partial x_i|, \tag{2}$$

a measure of local *sensitivity*.

Integrated Gradients (IG) [21] integrates the gradient along the straight path from x' to x:

$$\mathrm{IG}_i(x) = (x_i - x_i') \int_0^1 \frac{\partial f\big(x' + \alpha(x - x')\big)}{\partial x_i}\, d\alpha. \tag{3}$$

IG satisfies *completeness*, $\sum_i \mathrm{IG}_i(x) = f(x) - f(x')$. The integral is approximated by a Riemann sum with n steps, and the *convergence delta* $\delta = |\sum_i \mathrm{IG}_i - (f(x) - f(x'))|$ quantifies the approximation error. Because of the factor $(x_i - x_i')$, IG measures *contribution relative to the baseline*.

Attention Rollout [1] combines head-averaged attention matrices $A^{(l)}$ of L layers with the identity, which accounts for residual connections:

$$\tilde{A} = \prod_{l=1}^{L} \mathrm{norm}\Big(\tfrac{1}{2}A^{(l)} + \tfrac{1}{2}I\Big), \tag{4}$$

where norm renormalizes rows to sum to one.

Masking (occlusion) [23] replaces a region R by the baseline and records

$$M_R(x) = \big|f(x) - f(x_{R \leftarrow x'})\big|, \tag{5}$$

which needs one forward pass per region and no gradients.

4 Method

4.1 Posterior estimators and attribution surrogates

We use two kinds of model (Fig. 1). The *BayesFlow posterior*, trained for SIR and the GRF, optimizes Eq. (1) and is evaluated with recovery and SBC; the latent-space projection (UMAP and t-SNE of summary vectors) is computed on its summary network. The *attribution surrogate* is a plain PyTorch summary network followed by a linear head, trained with mean-squared error to predict all parameters, for all four simulators. A wrapper exposes a single output $f(x) = \hat{\theta}_k(x)$ for the target parameter k, as required by Captum [12]. The surrogate is needed because the flow's output is a distribution, not a differentiable scalar. **All attribution maps therefore describe what a network of the given architecture learns to extract from the data, not the normalizing-flow posterior itself.** Algorithm 1 summarizes the procedure.

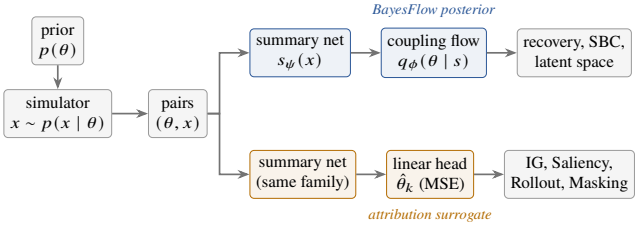


Figure 1: Pipeline. Every simulator trains a surrogate regressor of the summary-network family, on which attribution methods are computed. For SIR and the GRF, a BayesFlow posterior is also trained and evaluated with the standard diagnostics.

Algorithm 1 Attribution analysis for a registered simulator

Require: simulator $\mathcal{S}$, summary-net builder $\mathcal{B}$, method $\mathcal{M}$, target index k

Simulate $\{(\theta^{(j)}, x^{(j)})\}_{j=1}^{n}$ with $\mathcal{S}$; hold out 10%
$g \leftarrow \mathcal{B}(\mathcal{S})$; $h \leftarrow$ linear layer
Train (g, h) by Adam on $\sum_j \|h(g(x^{(j)})) - \theta^{(j)}\|^2$
$f(x) \leftarrow [h(g(x))]_k$
for each validation input x **do**
    $a(x) \leftarrow \mathcal{M}(f, x, x')$ ▷ IG, Saliency, Rollout or Masking
end for
Aggregate $|a|$ over samples, channels or time; plot

4.2 Registry

The package `bayesflow_xai.registry` holds four dictionaries of simulators, summary networks, inference networks and attribution methods, populated by decorators. A simulator is described by its parameter names, input kind (time series or image), a dataset builder and shape information. A YAML file selects one entry from each registry; `scripts/run_workflow.py` trains the surrogate (Adam, learning rate 10^{-3}) and runs the selected method. Adding a simulator or method requires only a decorated function.

4.3 Simulators

SIR epidemic. The stationary SIR model of the BayesFlow tutorial [16], in discrete time with population $N = 83 \cdot 10^6$:

$$\begin{aligned} C_t &= \lambda I_t S_t / N, & S_{t+1} &= S_t - C_t, \\ I_{t+1} &= I_t + C_t - \mu I_t, & R_{t+1} &= R_t + \mu I_t. \end{aligned} \tag{6}$$

Reported cases over $T = 60$ days start after a delay D and follow a negative binomial with mean C_t and dispersion ψ. Priors: $\lambda \sim \mathrm{LogN}(\log 0.4, 0.5)$, $\mu \sim \mathrm{LogN}(\log \frac{1}{8}, 0.2)$, $D \sim \mathrm{LogN}(\log 8, 0.2)$, $I_0 \sim \Gamma(2, 20)$, $\psi \sim$ Exp with mean 5. The

Table 1: Summary networks and training budgets of the attribution surrogates (and the BayesFlow posteriors).

Simulator	Summary network	Sims	Epochs
SIR	GRU (64 units, 8-dim)	6000	60
	Transformer (d=32, 4 heads, 2 layers)	4000	60
	BayesFlow: GRU + spline flow	*6000*	*100*
Toy signal	GRU (64 units, 8-dim)	6000	60
GRF	CNN (widths 8–64, 6-dim)	2000	30
	BayesFlow: CNN + coupling flow	*online*	*20*
GW	dilated CNN, 13 layers [10]	5000	40

posterior sees $\log(1 + \text{cases})$; the surrogates see the noise-free fractions $(S, I, R)/N \in \mathbb{R}^{60\times 3}$ and regress $\log(1 + \theta)$.

Toy signal ("Lotka–Volterra-style"). Despite its name, not the predator–prey ODE: with $\theta \sim \mathcal{U}(-1, 1)^3$ and 20 time points $t \in [0, 1]$,

$$X_k(t) = \theta_{k-1} \sin\big(2\pi t(1 + \theta_2)\big) + 0.05\,\varepsilon, \quad k \in \{1, 2\}. \quad (7)$$

Thus θ_0 affects only X_1: a ground truth for attribution.

Gaussian random field (GRF). A 64×64 field with power spectrum $P(k) = k^{-\alpha}\sigma^2$, $\log\sigma \sim \mathcal{N}(0, 1)$, $\alpha \sim \mathcal{N}(4, 0.5)$, from the BayesFlow spatial-data example. The repository also reproduces the example's conditional diffusion model that generates fields.

Gravitational wave (GW). The original benchmark simulator of the SBI practical guide [6], vendored unchanged: a binary black-hole merger (PyCBC, IMRPhenomPv2) with $m_1 \sim \mathcal{U}(40, 80)$ and $q = m_2/m_1 \sim \mathcal{U}(0.25, 0.99)$, projected onto the LIGO H1 and L1 detectors, with simulated noise, whitened and high-passed at 20 Hz; 4 s at 2048 Hz (8192 samples per detector), aligned so that the H1 merger is at $t = 0$. The target is m_1.

4.4 Architectures and settings

Table 1 lists the networks and training budgets. IG uses a zero baseline with $n = 50$ steps for SIR, the toy signal and the GRF, and the zero-strain level (the median input) with $n = 200$ for GW. Masking occludes single time steps (all channels) or 8×8 patches with zeros; for GW it occludes 32-sample (15.6 ms) windows of one detector at a time. For the SIR and toy-signal statistics the first and last time step are excluded from importance rankings (Sec. 6).

5 Experiments and Results

All numbers below are taken from the figures, logs and summaries in the repository. Training times on one NVIDIA RTX 2080 Ti were 8.9 min (SIR posterior), 44 min (GRF posterior) and 89 min (GRF diffusion model).

5.1 Posterior quality

For SIR (Fig. 2), recovery is near-perfect for the infection rate λ ($r = 0.98$), good for ψ (0.86) and I_0 (0.74), and weak for μ (0.18) and D (0.41), whose posteriors remain wide, as expected for weakly identified parameters. All SBC histograms lie inside the confidence band. The UMAP and t-SNE embeddings of 2000 summary vectors form a one-dimensional curve ordered by λ. For the GRF, recovery is near-perfect ($r = 0.999$ for $\log\sigma$, 0.993 for α) and the ECDF calibration curves stay inside the 95% band.

Table 2: Held-out R^2 of the GRU surrogates.

SIR	λ	μ	D	I_0	ψ
	0.90	−1.16	0.02	0.03	−0.01
Toy signal	θ_0	θ_1	θ_2		
	0.98	**0.99**	**0.97**		

Table 3: SIR, target λ: channel ranking and temporal focus per method.

Method	Channel ranking	Temporal focus
IG (200 samples)	$S \gg R > I$ (0.39/0.06/0.02)	days 0–3, 50–58
IG statistics (50)	S dominant in 48/50	days 1–2, 17–25, 54–59
Saliency	$R > S \gg I$ (0.16/0.13/0.03)	–
Masking	–	rising to last day
Attention rollout	–	days 49–53, last day

5.2 Surrogate fit

Attributions are only meaningful for parameters the surrogate predicts. Table 2 reports held-out R^2, recomputed on CPU with the repository's code and seeds. The SIR surrogate learns only λ: its targets are not standardized, so the loss is dominated by I_0 and ψ (variance ≈0.6 each on the $\log(1 + \cdot)$ scale), and ψ is not identifiable from noise-free trajectories at all. All SIR explanations therefore target λ.

5.3 Q1: Recovering a known ground truth

For θ_0, IG assigns the larger total attribution to X_1 in **188 of 200** samples (Fig. 3). The most important interior time steps cluster at steps 3–6 and 13–17, around the extrema of the sine, where the amplitude is easiest to read.

5.4 Q2: Agreement between methods on SIR

Table 3 and Fig. 4 show that the methods disagree. Paired t-tests on per-sample channel importance under IG are all significant ($p < 10^{-27}$), so the dominance of S is not noise. Saliency, which lacks the factor $(x_i - x_i')$ of Eq. (3), ranks R first. Since $S/N \approx 1$ over most of the window while I/N is a few percent, the zero baseline gives S a far larger multiplier; the S-dominance under IG is therefore at least partly a baseline effect [20]. The Transformer is trained on all five parameters jointly, so its rollout is identical for every target.

GRF. Mean IG and Saliency maps for α are diffuse over the interior with a dark frame of a few pixels, and 8×8 masking assigns the lowest importance to the outer patch columns. This matches the global role of α (spatial correlation), and the dark frame is consistent with zero padding, which gives border pixels fewer receptive fields.

5.5 Q3: Attribution exposes a simulator artefact

Using identical simulations (5000, data seed 0) and training, we vary only the input normalization (Table 4, Fig. 5). With the original min-max scaling, IG concentrates in blocks near −0.7, −0.2 and +0.3 s, where the merger contributes almost nothing; no sample has its attribution peak within ±50 ms of the merger.

Tracing the simulator stage by stage locates the cause.

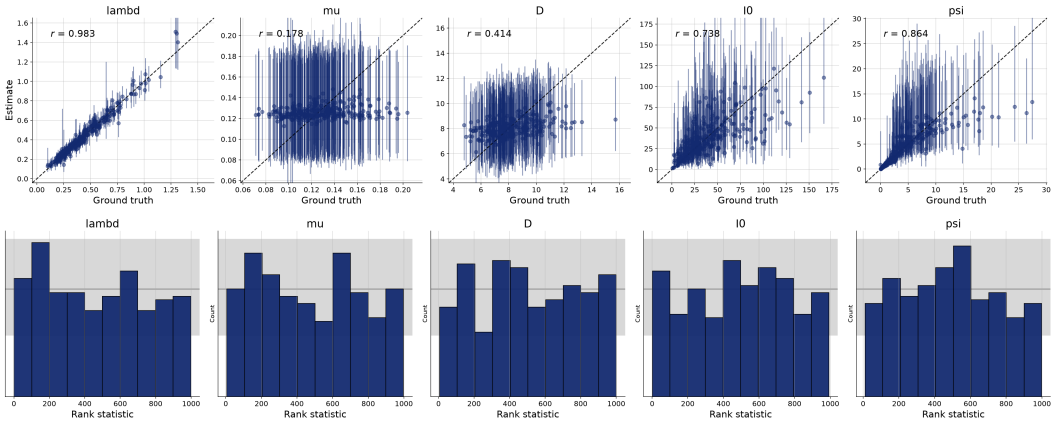


Figure 2: BayesFlow posterior for SIR on 300 held-out data sets (1000 draws each). **Top:** posterior median versus ground truth with posterior intervals. **Bottom:** SBC rank histograms; bars inside the grey band are consistent with calibration.

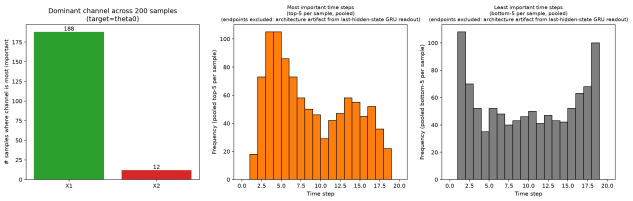


Figure 3: Toy signal, IG for target θ_0 over 200 validation samples. **Left:** dominant channel per sample; θ_0 scales only X_1 by construction. **Middle/right:** pooled top-5 and bottom-5 interior time steps per sample.

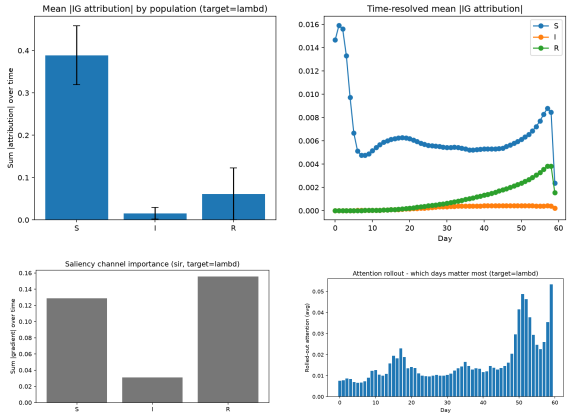


Figure 4: SIR, target λ. **Top:** IG by channel and over time. **Bottom left:** Saliency by channel. **Bottom right:** attention rollout per day (Transformer).

PyCBC starts the waveform below 20 Hz, where the noise power spectral density (PSD) is zero. Whitening estimates the PSD from the data and therefore *amplifies* the sub-20 Hz inspiral: the standard deviation at -1 s grows from 33 (whitened noise alone) to $2.5 \cdot 10^5$. The 20 Hz FIR high-pass reduces it only about sixfold. Every sample is dominated by a nearly monochromatic ∼20 Hz oscillation with a bell-shaped envelope from -1.5 to $+0.5$ s that continues past the merger, while the physical chirp is visible only in the last ∼50 ms. Min-max scaling devotes almost the full $[0, 1]$ range to this artefact. The simulator was deliberately left unmodified: the artefact is a finding of the attribution analysis: the higher validation error shows that the min-max model is worse, but not what it relies on.

Table 4: GW, target m_1: effect of input normalization (64 validation samples).

Normalization	Val. MSE	IG peak	Peak ±50 ms	Last 0.2 s
min-max (orig.)	9.27	−212 ms	**0%**	20%
z-score	**1.41**	−4.9 ms	**100%**	40%

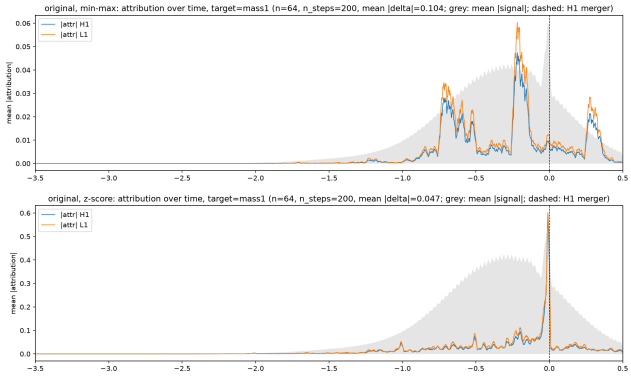


Figure 5: GW, mean |IG| over time (s, relative to the H1 merger) for min-max (top) and z-score inputs (bottom); grey: mean |signal|; dashed: merger. With min-max inputs the attribution avoids the merger.

5.6 Comparing methods on one model

On a z-score model (data seed 42, validation MSE 0.60), we compute IG, Saliency and Masking on the same 64 samples (Table 5). The rank correlation of the time profiles is 0.99 between IG and Masking and 0.47 for both pairs involving Saliency. IG's mean convergence delta is 0.046, against a mean $|f(x) - f(x')|$ of 12.3. Saliency's peaks in the signal-free window fall on a regular 0.25 s grid (512 samples = 2^9, one of the embedding's dilations), suggesting an architectural origin; this is an observation, not a tested claim.

6 Discussion

Attribution complements performance metrics. The validation error of the min-max GW model is higher than that of the z-score model, but it does not show why. The attribution analysis showed that the model ignored the merger, and following the attribution back into the simulator located the cause (Q3). Attribution addresses *what* a network learned, a question that error metrics do not answer. Because no posterior was trained for GW, we cannot say

Table 5: GW, one z-score model: IG vs. Saliency vs. Masking. Signal energy H1/L1: 45/55.

	IG	Saliency	Masking
Peak of mean attribution	−5.4 ms	−3010 ms	−10.7 ms
Peaks within ±50 ms	97%	25%	100%
Share in last 0.2 s	46%	11%	41%
Share in $[-3.5, -1]$ s	7.5%	58%	7.8%
Share after merger	13%	17%	14%
H1/L1 attribution	57/43	60/40	59/41

whether recovery or SBC would have flagged this model.

Agreement is more informative than a single map. IG and Masking agree almost perfectly on the GW model although one integrates gradients and the other uses only forward passes. Both measure change relative to the zero-strain baseline, so empty regions score near zero, whereas Saliency measures sensitivity, which can be high where there is no signal—in line with known failure modes of gradient maps [2, 11]. Where methods disagree, as on SIR (Q2), no single map should be trusted.

Fingerprints of architecture and baseline. Several patterns stem from the model or the method rather than the data. The GRU surrogates read only the final hidden state, so the last time step reaches the output through a single update and dominates attributions (according to the code documentation, it is the arg-max in every sample under both a zero and a mean baseline); endpoints are therefore excluded from the rankings. Zero baselines favour large-magnitude channels under IG; zero padding darkens image borders; and the dilated GW embedding leaves a periodic grid in Saliency.

Detector preference is not robust. The H1 preference of 57–60% is consistent across methods for one model, but an earlier z-score model (data seed 0) gave H1/L1 = 46/54, close to its signal energy (42/58). It is a property of a training run, not a physical result.

7 Limitations and Future Work

- **Surrogates, not posteriors.** Attributions are computed on MSE regressors, not on the BayesFlow summary network and flow; for SIR the surrogate also sees noise-free S/I/R instead of noisy cases.
- **No posterior for two simulators.** BayesFlow posteriors, and hence recovery and SBC, exist only for SIR and the GRF, not for the toy signal or GW.
- **Partial fit.** The SIR surrogate explains only λ; standardizing targets would likely help for μ and D.
- **Single runs.** Most results use one seed; the two z-score GW models differ by more than 2× in validation MSE.
- **No faithfulness metrics.** Methods are compared with each other and with known structure, not with deletion/insertion curves or randomization tests [2].
- **Baselines.** IG used a zero baseline for SIR, the toy signal and the GRF.

Future work includes training a GW posterior to test whether recovery and SBC detect the artefact, attributing through the BayesFlow summary network itself (e.g. to posterior moments), multi-seed studies, quantitative faithfulness evaluation, and systematic baseline comparisons.

8 Conclusion

Attribution methods are a useful complement to performance metrics in SBI. On a toy problem they recover the known structure; on a realistic gravitational-wave benchmark, Integrated Gradients revealed that a network relied on a simulator artefact rather than the physical signal, leading to a better normalization. Attributions also depend on architecture, baseline and training run, and are best read through agreement between methods.

Reproducibility. Code, configurations and figures are available at `https://github.com/ferihabircan/bayesflow-xai`. During preparation of this report, a bug in the SIR recovered-compartment update (recoveries computed from the updated rather than the previous infected count) was fixed; it affected only the trajectories seen by the surrogates, and the SIR surrogate figures were regenerated with unchanged qualitative results.

References

[1] S. Abnar and W. Zuidema. Quantifying attention flow in transformers. In *ACL*, 2020.

[2] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim. Sanity checks for saliency maps. In *NeurIPS*, 2018.

[3] M. Ancona, E. Ceolini, C. Öztireli, and M. Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *ICLR*, 2018.

[4] K. Cranmer, J. Brehmer, and G. Louppe. The frontier of simulation-based inference. *PNAS*, 117(48), 2020.

[5] M. Dax, S. R. Green, J. Gair, J. H. Macke, A. Buonanno, and B. Schölkopf. Real-time gravitational wave science with neural posterior estimation. *Physical Review Letters*, 127, 2021.

[6] M. Deistler et al. Simulation-based inference: A practical guide. arXiv preprint, 2025. Code: `https://github.com/sbi-dev/sbi-practical-guide`.

[7] L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using Real NVP. In *ICLR*, 2017.

[8] C. Durkan, A. Bekasov, I. Murray, and G. Papamakarios. Neural spline flows. In *NeurIPS*, 2019.

[9] D. Greenberg, M. Nonnenmacher, and J. Macke. Automatic posterior transformation for likelihood-free inference. In *ICML*, 2019.

[10] J. Hermans, A. Delaunoy, F. Rozet, A. Wehenkel, V. Begy, and G. Louppe. A trust crisis in simulation-based inference? Your posterior approximations can be unfaithful. *TMLR*, 2022.

[11] P.-J. Kindermans, S. Hooker, J. Adebayo, M. Alber, K. T. Schütt, S. Dähne, D. Erhan, and B. Kim. The (un)reliability of saliency methods. In *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*, Springer, 2019.

[12] N. Kokhlikyan et al. Captum: A unified and generic model interpretability library for PyTorch. arXiv:2009.07055, 2020.

[13] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *NeurIPS*, 2017.

[14] G. Papamakarios and I. Murray. Fast ϵ-free inference of simulation models with Bayesian conditional density estimation. In *NeurIPS*, 2016.

[15] S. T. Radev, U. K. Mertens, A. Voss, L. Ardizzone, and U. Köthe. BayesFlow: Learning complex stochastic models with invertible neural networks. *IEEE TNNLS*, 33(4), 2022.

[16] S. T. Radev, F. Graw, S. Chen, N. T. Mutters, V. M. Eichel, T. Bärnighausen, and U. Köthe. OutbreakFlow: Model-based Bayesian inference of disease outbreak dynamics with invertible neural networks and its application to the COVID-19 pandemics in Germany. *PLOS Computational Biology*, 17(10), 2021.

[17] S. T. Radev, M. Schmitt, L. Schumacher, L. Elsemüller, V. Pratz, Y. Schälte, U. Köthe, and P.-C. Bürkner. BayesFlow: Amortized Bayesian workflows with neural networks. *Journal of Open Source Software*, 8(89), 2023.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin. "Why should I trust you?" Explaining the predictions of any classifier. In *KDD*, 2016.

[19] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.

[20] P. Sturmfels, S. Lundberg, and S.-I. Lee. Visualizing the impact of feature attribution baselines. *Distill*, 2020.

[21] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *ICML*, 2017.

[22] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman. Validating Bayesian inference algorithms with simulation-based calibration. arXiv:1804.06788, 2018.

[23] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional networks. In *ECCV*, 2014.